

Tecnológico de Monterrey

Escuela de Ingeniería y Ciencias

Gramáticas Libres de Contexto

Parser para Mini-Triton

TC3002B — Computer Science Advanced Applications Development

Actividad 3.2: Gramáticas Libres de Contexto

Autores:	César Alán Silva Ramos — A01252916
	José María Soto Valenzuela — A01254831
	José Pablo Fong Coronado — A01252402
	Julián Enrique Espinoza Valenzuela — A01254679
	Víctor Jaziel Coronado Flores — A01644090
Profesor:	Dr. Adolfo Ernesto Arroyo Alanís
Campus:	Guadalajara
Fecha:	Mayo 2026

Tabla de Contenidos

1. [Gramática del lenguaje Mini-Triton](#)
2. [Decisiones de diseño](#)
3. [Precedencia y asociatividad](#)
4. [Diseño del AST](#)
5. [Tokenización asumida](#)
6. [Casos de prueba y resultados](#)
7. [Instrucciones de ejecución](#)

1. Gramática del lenguaje Mini-Triton

La gramática que define el lenguaje Mini-Triton está escrita en notación EBNF (Extended Backus-Naur Form). Cada regla de producción describe cómo se construyen las distintas partes de un programa válido. Se eligió EBNF porque permite expresar la repetición y las partes opcionales de forma directa con `*`, `+` y `?`, lo cual facilita la lectura y la traducción a código.

1.1 Gramática completa en EBNF

```
Program      → Decorator Kernel EOF

Decorator    → '@' 'triton' '.' 'jit'

Kernel       → 'def' ID '(' ParamList ')' ':' '{' StmtList '}'

ParamList    → Param ( ',' Param )*
              | ε

Param        → ID ( ':' QualifiedID )?

StmtList     → Stmt*

Stmt         → Assign
              | ExprStmt

Assign       → ID '=' Expr ';'

ExprStmt     → Expr ';'

Expr         → Term ( ( '+' | '-' ) Term )*

Term         → Factor ( ( '*' | '/' ) Factor )*

Factor       → NUMBER
              | '(' Expr ')'
              | NameOrCall

NameOrCall   → QualifiedID ( '(' ArgList ')' )?

QualifiedID  → ID ( '.' ID )*
```

```
ArgList    → Expr ( ',' Expr )*
           | ε
```

1.2 Explicación de las reglas

Program es el símbolo inicial. Todo archivo Mini-Triton contiene exactamente un decorador seguido de la definición de un kernel.

Decorator representa la secuencia fija `@triton.jit`. No es una regla genérica que acepte cualquier decorador; se verifica que el nombre sea literalmente "triton" y el método "jit".

Kernel define la estructura completa de una función: la palabra reservada `def`, el nombre de la función, los parámetros entre paréntesis, dos puntos, y el cuerpo encerrado en llaves.

ParamList y **Param** permiten cero o más parámetros separados por comas. Cada parámetro puede tener opcionalmente una anotación de tipo (por ejemplo `BS: tl.constexpr`).

StmtList es una secuencia de cero o más sentencias dentro del bloque `{ }`.

Stmt distingue entre dos tipos de sentencia: asignación (`id = expr;`) y sentencia de expresión (`expr;`). La distinción se hace observando los dos primeros tokens: si el primero es un ID y el segundo es `=`, es asignación; de lo contrario, es sentencia de expresión.

Expr y **Term** implementan la precedencia de operadores mediante niveles jerárquicos. **Expr** maneja `+` y `-` (menor precedencia), **Term** maneja `*` y `/` (mayor precedencia), y **Factor** maneja las unidades atómicas (números, paréntesis, nombres y llamadas).

NameOrCall permite identificadores simples (`x`), nombres cualificados con puntos (`tl.load`), y llamadas a función (`tl.load(x + offs)`). Si después del nombre cualificado hay un paréntesis, se interpreta como llamada; si no, como referencia a variable.

QualifiedID produce nombres con puntos como `tl.program_id`, `tl.arange`, `tl.constexpr`. Internamente se concatenan como un solo string.

ArgList permite cero o más expresiones separadas por comas dentro de una llamada a función. Solo soporta argumentos posicionales, ya que los argumentos nombrados (`mask=...`) están fuera del alcance.

2. Decisiones de diseño

2.1 Bloques con llaves en lugar de indentación

La especificación establece que los bloques se delimitan con `{ }` en lugar de usar la indentación de Python. Esto simplifica significativamente el lexer y el parser porque no es necesario implementar lógica de niveles de indentación, pilas de INDENT/DEDENT ni sensibilidad al espacio en blanco. Las llaves

proporcionan delimitadores explícitos que el parser puede reconocer con una regla simple de `match(LBRACE)` y `match(RBRACE)`.

2.2 Sentencias terminadas con punto y coma

Cada sentencia termina con `;`, lo cual elimina la ambigüedad sobre dónde termina una sentencia. En Python real, el fin de línea actúa como terminador, pero eso requiere que el lexer genere tokens de `NEWLINE` y que el parser sea sensible a ellos. El punto y coma es un terminador explícito que simplifica el parsing.

2.3 Distinción entre asignación y sentencia de expresión

Para distinguir `y = x + 1;` (asignación) de `tl.store(x);` (sentencia de expresión), el parser examina los dos primeros tokens:

1. Si el token actual es un `ID` y el siguiente es `=`, se parsea como asignación.
2. En cualquier otro caso, se parsea como sentencia de expresión.

Esta técnica se conoce como lookahead de 1 token adicional (LL(2) en ese punto de decisión). Es la recomendación explícita del documento de la tarea.

2.4 Nombres cualificados con punto

Los nombres como `tl.load`, `tl.program_id` o `tl.constexpr` se parsean mediante la regla `QualifiedID → ID ( '.' ID )*`. En lugar de tratar el punto como un operador binario de acceso a miembros, se construye un solo string concatenado. Esto simplifica el AST y refleja que en Mini-Triton los nombres con punto funcionan como identificadores compuestos, no como expresiones de acceso a atributos.

2.5 Eliminación de recursión izquierda

Las reglas de `Expr` y `Term` podrían escribirse con recursión izquierda directa:

```
Expr → Expr '+' Term | Expr '-' Term | Term
```

Sin embargo, un parser de descenso recursivo no puede procesar recursión izquierda porque entraría en un bucle infinito. La solución estándar es reescribir la regla usando iteración:

```
Expr → Term ( ( '+' | '-' ) Term )*
```

El resultado es equivalente en los programas que acepta, y la asociatividad por la izquierda se preserva al construir el AST de forma iterativa (el resultado parcial se va acumulando en la variable `left`).

2.6 Lenguaje de implementación

Se eligió Python 3 porque no requiere dependencias externas, es fácil de leer y ejecutar, y permite concentrar el esfuerzo en la lógica del parser en lugar de los detalles del lenguaje de implementación. El programa se ejecuta directamente con `python3` sin necesidad de instalar nada adicional.

3. Precedencia y asociatividad

La precedencia de operadores se resuelve mediante la jerarquía de no terminales en la gramática. Cada nivel de la gramática corresponde a un nivel de precedencia, donde los niveles más profundos se evalúan primero.

3.1 Tabla de precedencia

Nivel	Operadores	No terminal	Asociatividad
1 (menor)	<code>+</code> , <code>-</code>	Expr	Izquierda
2	<code>*</code> , <code>/</code>	Term	Izquierda
3 (mayor)	<code>()</code> , números, nombres, llamadas	Factor	N/A

3.2 Cómo funciona

Cuando el parser evalúa la expresión `a + b * c` , ocurre lo siguiente:

- `parse_expr()` llama a `parse_term()` para obtener el lado izquierdo.
- `parse_term()` llama a `parse_factor()` y obtiene `Name(a)` . No hay `*` ni `/` , así que regresa `Name(a)` .
- `parse_expr()` ve el `+` y consume el operador.
- `parse_expr()` llama de nuevo a `parse_term()` para el lado derecho.
- `parse_term()` llama a `parse_factor()` y obtiene `Name(b)` . Ve el `*` , lo consume, llama a `parse_factor()` de nuevo y obtiene `Name(c)` . Construye `BinaryOp("*", Name(b), Name(c))` .
- `parse_expr()` construye `BinaryOp("+", Name(a), BinaryOp("*", Name(b), Name(c)))` .

El resultado es que `*` se evalúa antes que `+` , que es el comportamiento correcto.

3.3 Asociatividad

La asociatividad por la izquierda se implementa mediante un ciclo `while` en lugar de recursión. Para la expresión `a - b - c` :

- `left = Name(a)`

2. Ve `-`, consume, parsea `Name(b)`. `left = BinaryOp("-", Name(a), Name(b))`
3. Ve `-`, consume, parsea `Name(c)`. `left = BinaryOp("-", BinaryOp("-", Name(a), Name(b)), Name(c))`

El resultado `(a - b) - c` refleja la asociatividad por la izquierda.

3.4 Paréntesis

Los paréntesis permiten al programador forzar un orden de evaluación diferente. Cuando el parser encuentra `(`, desciende recursivamente a `parse_expr()`, lo cual “reinicia” la precedencia al nivel más bajo. Así, `(a + b) * c` produce `BinaryOp("*", BinaryOp("+", Name(a), Name(b)), Name(c))`.

4. Diseño del AST

El árbol sintáctico abstracto (AST) representa la estructura del programa de forma que descarta los detalles sintácticos irrelevantes (llaves, punto y coma, paréntesis) y conserva solamente la estructura lógica.

4.1 Tipos de nodo

Nodo	Campos	Descripción
Program	kernel	Nodo raíz. Contiene un único kernel.
Kernel	name, params, body	Definición de función con nombre, lista de parámetros y lista de sentencias.
Param	name, annotation?	Parámetro con nombre y anotación de tipo opcional.
Assign	name, expr	Sentencia de asignación.
ExprStmt	expr	Expresión usada como sentencia (por ejemplo, una llamada a función).
BinaryOp	op, left, right	Operación binaria (suma, resta, multiplicación, división).
Call	name, args	Llamada a función con nombre y lista de argumentos.
Name	value	Referencia a una variable o identificador.
Number	value	Literal numérico (entero o flotante).

4.2 Jerarquía de clases

ASTNode (base abstracta)

- |— Program
- |— Kernel
- |— Param
- |— Assign
- |— ExprStmt
- |— BinaryOp
- |— Call
- |— Name
- |— Number

Todos los nodos heredan de `ASTNode` e implementan el método `pretty()` para generar una representación textual legible del árbol.

4.3 Ejemplo de AST

Para el programa:

```
@triton.jit
def add(x, y): {
    z = x + y;
}
```

El AST generado es:

```
Kernel(name="add", params=[x, y])
  body:
    Assign(z, BinaryOp("+", Name(x), Name(y)))
```

Para el programa más complejo:

```
@triton.jit
def k(x, out, BS: tl.constexpr): {
    pid = tl.program_id(0);
    offs = pid * BS + tl.arange(0, BS);
    tl.store(out + offs, tl.load(x + offs));
}
```

El AST generado es:

```

Kernel(name="k", params=[x, out, BS: tl.constexpr])
  body:
    Assign(pid, Call("tl.program_id", [Number(0)]))
    Assign(offsets, BinaryOp("+", BinaryOp("*", Name(pid), Name(BS)), Call("tl.arange", [Number(
      ExprStmt(Call("tl.store", [BinaryOp("+", Name(out), Name(offsets)), Call("tl.load", [Binary0

```

5. Tokenización asumida

El parser espera que el lexer produzca los siguientes tipos de token:

Tipo de token	Lexema	Descripción
AT	@	Inicio del decorador
DOT	.	Separador en nombres cualificados
COMMA	,	Separador de parámetros y argumentos
COLON	:	Separador en definición de kernel y anotaciones
SEMICOLON	;	Terminador de sentencias
ASSIGN	=	Operador de asignación
PLUS	+	Operador de suma
MINUS	-	Operador de resta
STAR	*	Operador de multiplicación
SLASH	/	Operador de división
LPAREN	(	Paréntesis de apertura
RPAREN	)	Paréntesis de cierre
LBRACE	{	Llave de apertura (inicio de bloque)
RBRACE	}	Llave de cierre (fin de bloque)
ID	[a-zA-Z_][a-zA-Z0-9_]*	Identificador
NUMBER	[0-9]+(\.[0-9]+)?	Literal numérico

Tipo de token	Lexema	Descripción
DEF	def	Palabra reservada
EOF	—	Fin de archivo

El lexer también ignora comentarios (que comienzan con `#`), espacios, tabulaciones y saltos de línea.

6. Casos de prueba y resultados

Se prepararon 13 casos de prueba: 7 programas válidos y 6 programas inválidos. Cada caso verifica un aspecto específico de la gramática.

6.1 Casos válidos

Caso 1: Suma simple (`01_valid_simple_add.mt`)

```
@triton.jit
def add(x, y): {
    z = x + y;
}
```

Verifica: estructura básica de un kernel con asignación y operación binaria.

Resultado:

```
VALIDO
Kernel(name="add", params=[x, y])
  body:
    Assign(z, BinaryOp("+", Name(x), Name(y)))
```

Caso 2: Kernel completo (`02_valid_full_kernel.mt`)

```
@triton.jit
def k(x, out, BS: tl.constexpr): {
    pid = tl.program_id(0);
    offs = pid * BS + tl.arange(0, BS);
    tl.store(out + offs, tl.load(x + offs));
}
```

Verifica: parámetros con anotación, llamadas a funciones cualificadas (`tl.program_id` , `tl.arange` , `tl.store` , `tl.load`), precedencia de `*` sobre `+` , sentencias de expresión.

Resultado:

```
VALIDO
Kernel(name="k", params=[x, out, BS: tl.constexpr])
  body:
    Assign(pid, Call("tl.program_id", [Number(0)]))
    Assign(offsets, BinaryOp("+", BinaryOp("*", Name(pid), Name(BS)), Call("tl.arange", [Number(
      ExprStmt(Call("tl.store", [BinaryOp("+", Name(out), Name(offsets)), Call("tl.load", [Binary0
```

Caso 3: Precedencia con paréntesis (`03_valid_precedence.mt`)

```
@triton.jit
def one(x): { y = (x + 1) * 2; }
```

Verifica: los paréntesis fuerzan que la suma se evalúe antes que la multiplicación.

Resultado:

```
VALIDO
Kernel(name="one", params=[x])
  body:
    Assign(y, BinaryOp("*", BinaryOp("+", Name(x), Number(1)), Number(2)))
```

Caso 4: Múltiples parámetros (`04_valid_multi_params.mt`)

```
@triton.jit
def f(a, b, c): { a = a + b * c; }
```

Verifica: tres parámetros y la precedencia natural de `*` sobre `+` sin paréntesis.

Resultado:

```
VALIDO
Kernel(name="f", params=[a, b, c])
  body:
    Assign(a, BinaryOp("+", Name(a), BinaryOp("*", Name(b), Name(c))))
```

Caso 5: Sentencia de expresión (05_valid_expr_stmt.mt)

```
@triton.jit
def g(x): { tl.load(x); }
```

Verifica: una llamada a función usada como sentencia (no como parte de una asignación).

Resultado:

```
VALIDO
Kernel(name="g", params=[x])
  body:
    ExprStmt(Call("tl.load", [Name(x)]))
```

Caso 6: Llamada anidada (06_valid_nested_call.mt)

```
@triton.jit
def h(x): { y = foo(x, 1, (2 + 3)); }
```

Verifica: llamada a función con múltiples argumentos, incluyendo una expresión entre paréntesis como argumento.

Resultado:

```
VALIDO
Kernel(name="h", params=[x])
  body:
    Assign(y, Call("foo", [Name(x), Number(1), BinaryOp("+", Number(2), Number(3))]))
```

Caso 7: Parámetro con anotación (07_valid_annotation.mt)

```
@triton.jit
def p(x, BS: tl.constexpr): { y = tl.arange(0, BS); }
```

Verifica: parámetro con anotación de tipo `tl.constexpr` y llamada a función cualificada.

Resultado:

```
VALIDO
Kernel(name="p", params=[x, BS: tl.constexpr])
  body:
    Assign(y, Call("tl.arange", [Number(0), Name(BS)]))
```

6.2 Casos inválidos

Caso 8: Falta el decorador (08_invalid_no_decorator.mt)

```
def missing_decorator(x): { y = x + 1; }
```

Por qué es inválido: todo programa Mini-Triton debe comenzar con `@triton.jit`. Sin el decorador, la gramática espera `@` como primer token pero encuentra `def`.

Resultado:

```
INVALIDO
se esperaba AT pero se encontro DEF ('def')
En: linea 1, columna 1
```

Caso 9: Falta el bloque con llaves (09_invalid_no_braces.mt)

```
@triton.jit
def bad_block(x): y = x + 1;
```

Por qué es inválido: el cuerpo del kernel debe estar encerrado en `{ }`. Sin la llave de apertura, el parser espera `{` pero encuentra `y`.

Resultado:

```
INVALIDO
se esperaba LBRACE pero se encontro ID ('y')
En: linea 2, columna 19
```

Caso 10: Falta el punto y coma (10_invalid_no_semicolon.mt)

```
@triton.jit
def bad_semi(x): {
    y = x + 1
}
```

Por qué es inválido: cada sentencia debe terminar con `;`. La expresión `y = x + 1` termina pero el parser no encuentra el `;` esperado antes de `}`.

Resultado:

```
INVALIDO
se esperaba SEMICOLON pero se encontro RBACE ('}')
En: linea 4, columna 1
```

Caso 11: Expresión incompleta (11_invalid_incomplete_expr.mt)

```
@triton.jit
def bad_expr(x): { y = x + ; }
```

Por qué es inválido: después de `+` se espera un operando (número, identificador o subexpresión), pero se encuentra `;` directamente. La expresión queda incompleta.

Resultado:

```
INVALIDO
se esperaba una expresion (numero, identificador o '('), pero se encontro SEMICOLON (';')
En: linea 3, columna 11
```

Caso 12: Argumentos nombrados (12_invalid_kwargs.mt)

```
@triton.jit
def kwarg(x): { y = tl.load(x, mask=1); }
```

Por qué es inválido: Mini-Triton no soporta argumentos nombrados (`mask=1`). El parser interpreta `mask` como una expresión, pero al encontrar `=` después (que no es un operador válido en ese contexto), falla.

Resultado:

```
INVALIDO
se esperaba RPAREN pero se encontro ASSIGN ('=')
En: linea 3, columna 22
```

Caso 13: Estructura de control (13_invalid_if_statement.mt)

```
@triton.jit
def control(x): { if (x) { y = 1; } }
```

Por qué es inválido: Mini-Triton no soporta sentencias `if` . El parser intenta interpretar `if` como un identificador (ya que no es palabra reservada en Mini-Triton), pero la estructura resultante no encaja en ninguna regla válida.

Resultado:

```
INVALIDO
se esperaba SEMICOLON pero se encontro LBRACE ('{')
En: linea 3, columna 10
```

6.3 Resumen de resultados

#	Archivo	Esperado	Obtenido	Correcto
1	01_valid_simple_add.mt	VALIDO	VALIDO	Si
2	02_valid_full_kernel.mt	VALIDO	VALIDO	Si
3	03_valid_precedence.mt	VALIDO	VALIDO	Si
4	04_valid_multi_params.mt	VALIDO	VALIDO	Si
5	05_valid_expr_stmt.mt	VALIDO	VALIDO	Si
6	06_valid_nested_call.mt	VALIDO	VALIDO	Si
7	07_valid_annotation.mt	VALIDO	VALIDO	Si
8	08_invalid_no_decorator.mt	INVALIDO	INVALIDO	Si
9	09_invalid_no_braces.mt	INVALIDO	INVALIDO	Si
10	10_invalid_no_semicolon.mt	INVALIDO	INVALIDO	Si
11	11_invalid_incomplete_expr.mt	INVALIDO	INVALIDO	Si
12	12_invalid_kwargs.mt	INVALIDO	INVALIDO	Si
13	13_invalid_if_statement.mt	INVALIDO	INVALIDO	Si

Los 13 casos producen el resultado esperado. Los programas válidos generan su AST completo, y los programas inválidos reportan un mensaje de error con la posición exacta del problema.

7. Instrucciones de ejecución

Requisitos

Python 3.6 o superior. No se requieren dependencias externas.

Ejecución

```
cd entrega/  
python3 mini_triton_parser.py <archivo.mt>
```

Ejemplos

```
# Programa valido: imprime VALIDO + AST  
python3 mini_triton_parser.py tests/01_valid_simple_add.mt  
  
# Programa invalido: imprime INVALIDO + error  
python3 mini_triton_parser.py tests/08_invalid_no_decorator.mt
```

Estructura del proyecto

```
entrega/  
├─ mini_triton_parser.py      <-- Punto de entrada  
├─ src/  
│   ├─ __init__.py  
│   ├─ lexer.py              <-- Analizador lexico  
│   ├─ parser.py             <-- Parser de descenso recursivo  
│   └─ ast_nodes.py          <-- Nodos del AST  
├─ tests/  
│   ├─ 01_valid_simple_add.mt  
│   ├─ 02_valid_full_kernel.mt  
│   ├─ 03_valid_precedence.mt  
│   ├─ 04_valid_multi_params.mt  
│   ├─ 05_valid_expr_stmt.mt  
│   ├─ 06_valid_nested_call.mt  
│   ├─ 07_valid_annotation.mt  
│   ├─ 08_invalid_no_decorator.mt  
│   ├─ 09_invalid_no_braces.mt  
│   ├─ 10_invalid_no_semicolon.mt  
│   ├─ 11_invalid_incomplete_expr.mt  
│   └─ 12_invalid_kwargs.mt
```

```
|  └─ 13_invalid_if_statement.mt
└─ reporte.md                <-- Este documento
```